

ATmega32A Module

Common Definitions (`common.hpp`) & Seven-Segment Display (`sevensseg.hpp`)

Ali Sahafi

1 What this file is

Every driver module in this course (GPIO, ADC, UART, ...) starts with `#include "../common/common.hpp"`. You never include this file yourself — it comes along automatically with whichever module you use. It provides three things:

1. The MCU definition `__AVR_ATmega32A__` so that `<avr/io.h>` exposes the correct register names.
2. The standard AVR headers: `<avr/io.h>` (registers), `<avr/interrupt.h>` (ISR support, `sei()/cli()`) and `<util/delay.h>` (`_delay_ms()`).
3. A set of Arduino-like constants used across all modules.

2 Constants

Constant	Value	Meaning
INPUT	0	Pin is an input (high impedance)
OUTPUT	1	Pin is an output
INPUT_PULLUP	2	Input with the internal pull-up resistor enabled
LOW	0	Logic 0 (GND)
HIGH	1	Logic 1 (VCC)
ALL	0xFF	Selects the <i>whole port</i> instead of one pin

3 The F_CPU rule

F_CPU tells the toolchain how fast your CPU clock is. Delay loops, UART baud rates and timer calculations are all derived from it, so it must match the *real* clock of your chip (set by the fuse bits — see the Makefile targets `int-1mhz ... ext-xtal`).

Always define it *before* the first driver include:

```
#define F_CPU 8000000UL // 8 MHz -- must come first!
#include "drivers/gpio/gpio.hpp"
```

If you forget, the driver falls back to 8MHz and the compiler prints a warning. A wrong F_CPU does not stop compilation — but your delays will be wrong and your UART will print garbage. If the serial terminal shows random characters, check F_CPU and the fuses first.

4 One source file per program

The drivers are *header-only*: all code lives in `.hpp` files and each firmware image is built from exactly one `.cpp` file. Include only the modules you need — unused peripherals then cost no flash memory and no interrupt vectors.

5 Seven-Segment Display (`sevensseg.hpp`)

`sevensseg.hpp` is a small helper module for driving a single **active-low, common-anode** seven-segment display that is wired directly to **PORTB**. Include it explicitly — it is not pulled in automatically by other modules:

```
#define F_CPU 8000000UL
#include "../common/sevensseg.hpp"
```

Wiring. All 8 bits of **PORTB** are dedicated to the display's segments (including the decimal point), so **PORTB** must not be used for anything else while this module is active. The driver sets **PORTB** directly; it does not touch **DDRB**, so you must configure **PORTB** as **OUTPUT** yourself (e.g. via `GPIO.setDirection(PORTB, ALL, OUTPUT)`) before calling it.

API reference.

Function	Description
<code>SevenSeg(digit)</code>	Displays <code>digit</code> (0–9) on the seven-segment display and returns the raw segment pattern written.

`digit` is expected to be in the range 0–9. Any other value (e.g. from an unmasked 4-bit input of 10–15) falls back to a dash-like pattern (0b10001100) instead of showing a digit, which is useful for spotting out-of-range input while debugging.

Example: binary switches to seven-segment digit.

```
#define F_CPU 8000000UL
#include "../gpio/gpio.hpp"
#include "../common/sevensseg.hpp"

int main() {
    GPIO.setDirection(PORTB, ALL, OUTPUT); // seven-segment on PORTB
    GPIO.setDirection(PORTC, ALL, INPUT_PULLUP); // DIP switches on PORTC

    while (1) {
        uint8_t value = GPIO.read(PORTC) & 0x0F; // lower 4 bits -> 0..15
        SevenSeg(value);
        _delay_ms(50);
    }
}
```